



# Diplomado en Data Science Aplicada con Python para la Toma de Decisiones

---

Clase 1: Fundamentos de Python + Control de Flujo



Carlos Enrique Mosquera Trujillo



SLB Ecuador

## Instructor

---



✉ cmosquerat@unal.edu.co

in cmosquerat56

### Carlos Enrique Mosquera Trujillo

Ingeniero Electrónico · M.Sc. Automatización Industrial

- 🎓 Ing. Electrónica y M.Sc. en Automatización Industrial, **Universidad Nacional de Colombia**.
- 🔌 **6+ años en Python y SQL:** pipelines de datos, automatización de reportes, analítica operativa.
- 🖥️ **Gestor de HPC (BIOS):** cómputo de alto rendimiento para cargas intensivas en datos. Administración de sistemas Linux.
- 👤 **Profesor UDLA:** Dos años de experiencia en diplomados. PySpark y Metaprogramación.

# Metodología: Project-Based Learning

Contexto operativo  
SLB Ecuador



Problema Real

Teoría mínima  
necesaria



Conceptos Clave

Google Colab  
en vivo



Código

Resultado medible  
y auditable



Solución

Siguiente  
iteración



Reflexión

## Cada sesión de 2h sigue este ciclo:

1. Se presenta un **problema operativo real** de un campo petrolero.
2. Se introducen los **conceptos** necesarios para resolverlo.
3. Se escribe **código en vivo** en Google Colab, juntos.
4. Se llega a una **solución funcional**: un reporte, una métrica.
5. Se **reflexiona**: ¿qué más se podría hacer? Eso motiva la siguiente clase.

# ¿Por Qué Aprender a Programar Hoy?

## La IA ya genera código y reportes.

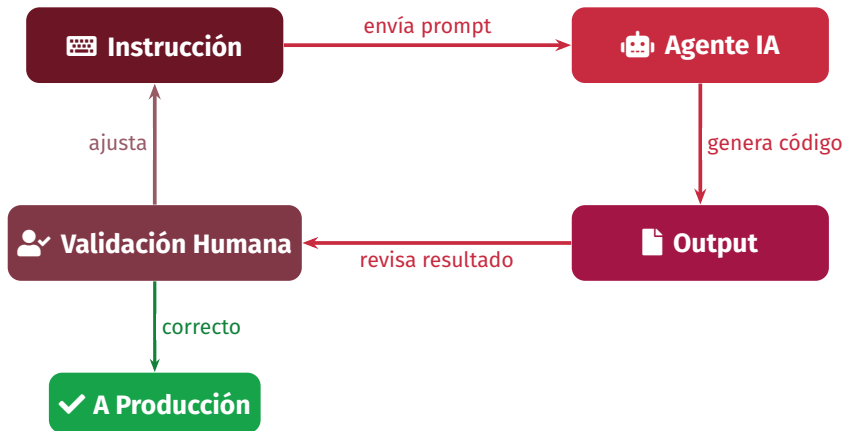
- 🗑️ ChatGPT, Copilot y Claude **escriben Python**, generan análisis y producen reportes completos en segundos.
- ⚠️ Pero **cometen errores**: calculan mal un corte de agua, confunden un pozo con otro, inventan datos que no existen.
- 👤 Quien **sabe leer código** detecta el error antes de que llegue a una decisión operativa. Quien no, lo publica como verdad.

## Programar no es competir con la IA. Es saber dirigirla.

- 🔍 **Auditar**: leer el código que la IA genera y verificar que la lógica sea correcta.
- ⚙️ **Ajustar**: modificar parámetros, corregir fórmulas, adaptar la solución al contexto real.
- 🔧 **Orquestar**: conectar múltiples herramientas de IA en un flujo de trabajo confiable.

*Este diplomado nos prepara para ese rol: el profesional que entiende, valida y decide.*

# Human-in-the-Loop



# ¿Por Qué Python?

---

**#1 en TIOBE, Stack Overflow y GitHub desde 2021.**



## Legible

Sintaxis clara: se lee casi como inglés técnico.

Menos código, menos errores.



## Ecosistema

pandas, numpy, lasio, scikit-learn.

Todo resuelto y documentado.



## Industrial

SLB, Shell, Chevron, ExxonMobil, Petrobras, NASA.

Petrofísica, producción, forecasting.



## Académico

Lenguaje de referencia en

MIT, Stanford y Harvard para cursos introductorios.

*Python no es un lenguaje académico. Es infraestructura de negocio.*

# Python en la Operación Diaria

---

Con este diplomado vamos a automatizar tareas como estas:



## Reportes

Leer 50 archivos LAS de well logs, consolidar en un dataset y exportar KPIs.

4 horas manual → 30 seg



## Análisis

Cruzar producción diaria con registros de sensores, detectar caídas de flujo.

Sin depender de TI



## IA Aplicada

Predecir producción por pozo, clasificar litologías y optimizar intervenciones.

Objetivo del diplomado

**No necesitamos ser programadores.** Necesitamos saber lo suficiente para dirigir, auditar y decidir.

# Entorno de Trabajo

Google Colab y repositorio del curso

# ¿Qué es Google Colab?

## Entorno de programación en la nube. Gratuito. Cero instalaciones.

-  Solo necesitamos una cuenta de Google y un navegador.
-  Organizado en **celdas**: código → ejecución → resultado.
-  Se comparte como un Google Doc. Trabajo colaborativo.
-  Google provee CPU, RAM y GPU sin costo.

```
[1] pozo = 'Sacha042'
```

```
[2] bopd = 1250
```

```
> Sacha042
```

```
[3] print(pozo)
```

```
> 1250
```

# Repositorio del Curso

**Todo el material centralizado. Un link, todo el curso.**

-  **Presentaciones:** slides de cada clase en PDF.
-  **Notebooks:** cuadernos Colab con ejercicios y micro-proyectos.
-  **Datos:** datasets petroleros para las prácticas.
-  **Versiónado:** cada clase se publica como nueva versión.

## Repositorio:

[github.com/cmosquerat/slb-diplomado](https://github.com/cmosquerat/slb-diplomado)

**slb-diplomado/**

clase-01/

clase-02/

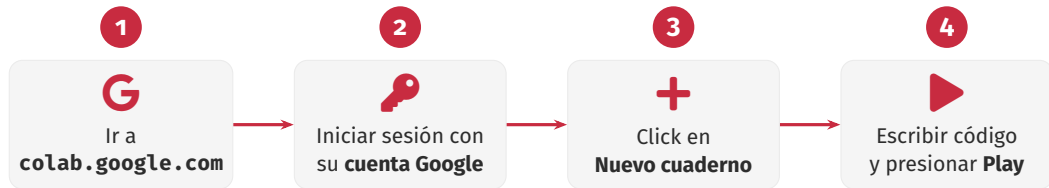
clase-03/

datos/

README.md

# Abrir Google Colab

---



**Hagámoslo ahora.** Abramos `colab.google.com` y creemos un cuaderno nuevo.

# Fundamentos de Python

Antes de escribir código

# ¿Qué es Programar?

Descomponer un problema en pasos que una máquina pueda ejecutar.

## 🏢 Problema operativo

*“¿Cuál es la utilidad neta del pozo Sacha-042 hoy?”*



**Paso 1:** Definir producción (BOPD)

```
bopd = 1250
```

**Paso 2:** Definir precio del crudo

```
precio = 72.50
```

**Paso 3:** Definir costo por barril

```
costo = 38.00
```

**Paso 4:** Calcular y mostrar utilidad

```
print(bopd * (precio - costo))
```

Una computadora no interpreta esta frase. No sabe qué es “utilidad”, ni dónde están los datos. **Necesitamos descomponerlo en pasos.**

**Programar = pensar con precisión.** El mismo proceso que ya hacemos en una hoja de cálculo, pero sin ambigüedad.

# Sintaxis: La Gramática de Python

Así como el español tiene reglas, Python tiene sintaxis.

- A Mayúsculas importan:** `print` no es `Print`.
- </> Paréntesis obligatorios:** toda función lleva `()`.
- ” Comillas para texto:** `"hola"` es texto, `hola` sin comillas es una variable.
- ☰ Indentación:** los espacios al inicio definen la estructura del código.

✓ **Correcto**

```
bopd = 1250  
print(bopd)
```

✗ **Errores comunes**

```
Print(bopd)    P mayúscula  
print(bopd    falta )  
print bopd    faltan ()
```

*Un solo carácter mal y Python nos avisa con un error.*

# print() y Operaciones

**print()** muestra un valor en pantalla. Python también sabe hacer cuentas.

▶ Escribamos esto en Colab

```
print("Hola SLB Ecuador")  
print(1250 * 72.50)  
print(1250 * (72.50 - 38.00))  
print(1250 ** 2)
```

>\_ Output

```
Hola SLB Ecuador  
90625.0  
43125.0  
1562500
```

| Op. | Significado        |
|-----|--------------------|
| + - | Suma, resta        |
| * / | Producto, división |
| **  | Potencia           |
| //  | División entera    |
| %   | Residuo (módulo)   |

Respetar el orden matemático: primero \* y /, después + y -.

**Shift + Enter** ejecuta la celda.

## Práctica Guiada

🕒 5 min

**El pozo Sacha-042 produce 1 250 barriles/día (BOPD). El crudo se vende a \$ 72.50/bbl.**

Usando solo `print()` y operaciones, calculemos en Colab:

1. El **ingreso bruto diario** del pozo.
2. Si el costo operativo es \$ 38.00/bbl, ¿cuál es la **utilidad diaria**?
3. ¿Cuál es el **margen operativo** en porcentaje?

💡 **Pista:**  $\text{Margen} = (\text{precio} - \text{costo}) / \text{precio} \times 100$

▶ Tres print, tres respuestas

```
print( ___ )  
print( ___ )  
print( ___ )
```

## Solución — Práctica Guiada

Python

```
print(1250 * 72.50)
print(1250 * 72.50 - 1250 * 38.00)
print((72.50 - 38.00) / 72.50 * 100)
```

>\_ Output

```
90625.0
43125.0
47.586206896551715
```

- ✓ Ingreso bruto: \$ 90 625/día.
- ✓ Utilidad diaria: \$ 43 125.
- ✓ Margen operativo: 47.6 %.
- ⚠ Escribimos 1250 y 72.50 **tres veces**. Si cambian, corregimos todo a mano.

# Variables y Tipos

Nombres, valores y conversiones

# Variables: Asignar y Reutilizar

Una variable es un nombre que apunta a un valor en memoria.

## Python

```
bopd = 1250
precio = 72.50
costo = 38.00
ingreso = bopd * precio
utilidad = ingreso - bopd * costo
print(ingreso)
print(utilidad)
```

## >\_ Output

```
90625.0
43125.0
```

- = no es "igual a". Significa **asignar**: valor de la derecha se guarda con el nombre de la izquierda.
- Si el precio del crudo sube a \$78, cambiamos **una sola línea** y todo se recalcula.
- ⚠ Precio  $\neq$  precio. Python distingue mayúsculas.

# Nombrar Bien Importa

---

Un buen nombre hace que el código se explique solo.

---

✓ **Correcto**

✗ **Evitar**

---

bopd\_pozo

b

precio\_crudo

p1

corte\_agua\_pct

dato

costo\_op\_bbl

x

---

Convención en Python: snake\_case

palabras en minúscula separadas por \_

## Reglas del lenguaje:

- ✓ Empieza con letra o \_
- ✓ Puede contener letras, números, \_
- ✗ No puede empezar con número
- ✗ No puede tener espacios ni -
- ✗ No usar palabras reservadas: print, if, for, True...

## Tipos de Datos

Python clasifica cada valor. El tipo determina qué operaciones son válidas.

|   | Tipo            | Ejemplo      | Uso en operación                   |
|---|-----------------|--------------|------------------------------------|
| # | int — entero    | 1250         | Barriles, número de pozos          |
| % | float — decimal | 72.50        | Precios, cortes de agua, porosidad |
| A | str — texto     | "Sacha-042"  | Nombre de pozo, campo, formación   |
| ○ | bool — lógico   | True / False | ¿Pozo activo? ¿Supera cutoff?      |

### Python

```
print(type(1250))
print(type(72.50))
print(type("Sacha-042"))
print(type(True))
```

# Cuidado al Mezclar Tipos

Sumar texto con número no funciona. Python nos avisa con un error.

✗ Esto falla

Python

```
print("BOPD: " + 1250)
```

>\_ Output

```
TypeError: can only concatenate  
str to str (not "int")
```

✓ Dos soluciones

Python

```
print("BOPD: " + str(1250))  
print("BOPD:", 1250)
```

>\_ Output

```
BOPD: 1250  
BOPD: 1250
```

💡 `str()`, `int()` y `float()` **convierten** entre tipos. Usaremos esto en la siguiente diapositiva con `input()`.

# input() — Leer Datos del Usuario

input() pausa el programa y espera a que el usuario escriba.



## Python

```
pozo = input("Pozo: ")
print(pozo)
print(type(pozo))
```

## >\_ Output

```
Pozo: Sacha-042
Sacha-042
<class 'str'>
```

- El texto entre paréntesis es el **mensaje** que ve el usuario.
- Lo que el usuario escribe se guarda en la variable (pozo) como **cadena de texto (str)**.
- ⚠ Aunque el usuario escriba 1250, Python lo guarda como el **texto** "1250", **no** como número.

# input() + Conversión: Para Operar con Números

Si el dato es numérico, envolvemos input() con int() o float().

## Python

```
pozo = input("Pozo: ")
bopd = int(input("BOPD: "))
precio = float(input("Precio ($/bbl): "))

ingreso = bopd * precio
print(pozo, "-> ingreso: $", ingreso)
```

## >\_ Output

```
Pozo: Sacha-042
BOPD: 1250
Precio ($/bbl): 72.5
Sacha-042 -> ingreso: $ 90625.0
```

- ≡ int(input(...)) se lee **de adentro hacia afuera**: primero input pide el dato, después int lo convierte.
- # int() para enteros (barriles, número de pozos).
- % float() para decimales (precio, corte de agua).
- ✓ Con esto ya podemos hacer programas que reciben datos de un ingeniero y responden con métricas.

# Control de Flujo

Que el programa decida y repita

# Comparaciones y Booleanos

Toda comparación devuelve True o False. Con eso el programa decide.

| Operador | Significado               |
|----------|---------------------------|
| ==       | Igual a                   |
| !=       | Distinto de               |
| > <      | Mayor / menor             |
| >= <=    | Mayor-igual / menor-igual |
| and      | Ambas ciertas             |
| or       | Al menos una cierta       |
| not      | Niega                     |

Python

```
bopd, corte = 1250, 0.32
print(bopd > 1000)
print(corte < 0.5)
print(bopd > 1000 and corte <
↪ 0.5)
```

>\_ Output

```
True
True
True
```

## Condicionales: if / elif / else

El programa toma un camino según el dato. La indentación define el bloque.

### Python

```
bopd = 1250

if bopd >= 1000:
    estado = "productor alto"
elif bopd >= 300:
    estado = "productor medio"
else:
    estado = "candidato a
    ↪ workover"

print("Sacha-042:", estado)
```

- El : al final de la línea abre un bloque.
- **4 espacios** de indentación agrupan las líneas que pertenecen al bloque.
- elif se puede repetir; else es opcional.
- ⚠ Solo se ejecuta **una** rama: la primera que sea True.

## Su Turno: Clasificador de Pozo

🕒 5 min

Escriban un programa que pida producción y corte de agua y clasifique al pozo.

Reglas de negocio (simplificadas):

- Si `bopd >= 1000` y `corte_agua < 0.5` → “**óptimo**”
- Si `bopd >= 300` → “**aceptable**”
- Si `corte_agua >= 0.9` → “**candidato a cierre**”
- En cualquier otro caso → “**revisión técnica**”

🔗 Python

```
bopd = int(input("BOPD: "))
corte_agua = float(input("Corte de agua (0-1): "))
# escriban el if / elif / else aquí
```

## Solución — Clasificador de Pozo

Python

```
bopd = int(input("BOPD: "))
corte_agua = float(input("Corte de agua:
↪ "))

if bopd >= 1000 and corte_agua < 0.5:
    estado = "optimo"
elif corte_agua >= 0.9:
    estado = "candidato a cierre"
elif bopd >= 300:
    estado = "aceptable"
else:
    estado = "revision tecnica"

print("Estado:", estado)
```

- ✓ El orden **importa**: la condición más restrictiva va primero.
- ✓ and combina condiciones.
- ⚠ Si moviéramos `bopd >= 300` arriba, el pozo óptimo nunca llegaría a "óptimo".

# Listas: Varios Valores, Un Nombre

Cuando no tenemos un pozo, sino una batería de pozos.

Python

```
pozos = ["Sacha-042", "Sacha-108",  
         "Shushufindi-21",  
         ↪ "Auca-63"]  
bopd = [1250, 780, 1600, 340]  
  
print(pozos[0])           # primer pozo  
print(bopd[-1])           # ultimo BOPD  
print(pozos[1:3])         # slice  
print(len(bopd))          # cuantos hay  
  
bopd.append(920)           # agregar  
print(sum(bopd))          # suma total
```

- Se crean con [ ] separando por comas.
- Se indexan desde 0. -1 es el último.
- lista[a:b] devuelve del a al b-1: **slice**.
- len() cuenta, sum() suma, append() agrega al final.
- **Mismo orden** entre pozos y bopd: la posición 0 es el mismo pozo en ambas listas.

## Ciclos: for — Repetir Sin Copiar-Pegar

**for** recorre una lista y ejecuta el mismo bloque para cada elemento.

Python

```
pozos = ["Sacha-042", "Sacha-108",  
         "Shushufindi-21",  
         ↪ "Auca-63"]  
bopd = [1250, 780, 1600, 340]  
  
for produccion in bopd:  
    print(produccion, "BOPD")  
  
# Recorrer dos listas en paralelo  
for pozo, prod in zip(pozos, bopd):  
    print(pozo, "->", prod)
```

>\_ Output

```
1250 BOPD  
780 BOPD  
1600 BOPD  
340 BOPD  
Sacha-042 -> 1250  
Sacha-108 -> 780  
Shushufindi-21 -> 1600  
Auca-63 -> 340
```

*zip() empareja dos listas posición por posición.*

## for + if: Filtrar y Contar

¿Cuántos pozos superan el cutoff de 1000 BOPD? ¿Cuál produce más?

Python

```
pozos = ["Sacha-042", "Sacha-108",  
         "Shushufindi-21", "Auca-63"]  
bopd = [1250, 780, 1600, 340]  
  
sobre_cutoff = 0  
total = 0  
  
for pozo, prod in zip(pozos, bopd):  
    total = total + prod  
    if prod >= 1000:  
        sobre_cutoff = sobre_cutoff + 1  
        print(pozo, "supera cutoff")  
  
print("Total:", total, "BOPD")  
print("Sobre cutoff:", sobre_cutoff)
```

>\_ Output

```
Sacha-042 supera cutoff  
Shushufindi-21 supera cutoff  
Total: 3970 BOPD  
Sobre cutoff: 2
```

Un acumulador (*total*) y un contador (*sobre\_cutoff*): el patrón que después haremos en pandas con una sola línea.

# Caso Integrador

Reporte diario de un pozo

## Reporte de Pozo Sacha-042

🕒 20 min

Antes del handover de las 7 PM, ingeniería de producción necesita un **reporte diario** del pozo Sacha-042 para el jefe de campo.

Del pozo se conoce:

- 💧 **Producción de líquido total:** 1840 barriles/día
- % **Corte de agua:** 32 %
- \$ **Precio del crudo Oriente:** \$ 72.50/bbl
- ⚙️ **Costo operativo (lifting cost):** \$ 38.00/bbl de líquido
- ❗ **Cutoff de rentabilidad definido por producción: 500 BOPD netos**

**El reporte debe entregar:**

1. **BOPD netos** (líquido total menos agua) y **BWPD**
2. **Ingreso, costo, utilidad diaria y margen %**
3. **Clasificación del pozo** con if / elif / else según BOPD y corte de agua
4. **Proyección mensual** de utilidad (30 días)

# Cierre

Recapitulación

# Lo Que Aprendimos Hoy

---

## Conceptos

- ✓ Qué es programar y por qué importa hoy
- ✓ Google Colab como entorno de trabajo
- ✓ Variables, tipos de datos y conversiones
- ✓ Comparaciones y condicionales
- ✓ Listas y ciclos for

## Habilidades

- ✓ Escribir y ejecutar código en Colab
- ✓ Calcular métricas de un pozo con Python
- ✓ Clasificar pozos con `if / elif / else`
- ✓ Recorrer una batería de pozos con `for`
- ✓ Producir un reporte diario ejecutivo

## Próxima Clase

---

**En la Clase 2 dejamos las listas y damos el salto al stack de data science.**

- ➔ **numpy** — arreglos numéricos rápidos: operar sobre miles de mediciones a la vez.
- ➔ **pandas** — DataFrames: la “batería de pozos” de hoy, con superpoderes.
- ➔ **Lectura de datos reales** — CSV, Excel y JSON de producción y sensores.
- ➔ **matplotlib y seaborn** — graficar producción, cortes de agua y correlaciones.

# ¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ [cmosquerat@unal.edu.co](mailto:cmosquerat@unal.edu.co)

Diplomado en Data Science Aplicada con Python

SLB Ecuador · UDLA · 2026